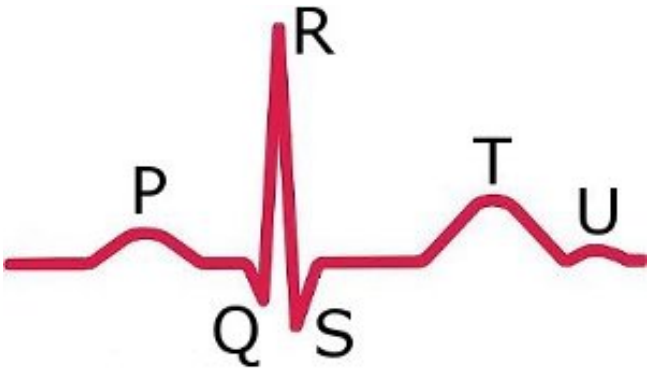
THE UNIVERSITY OF MELBOURNE

Final Report

Respiratory Rate Estimation via ECG data

Author:
Luke Kuflik (1463828)

Subject Handbook code:
COMP-90072



Source: `https://simulide.com/p/storage/2023/11/ecg_signal.jpg`

Faculty of Science
The University of Melbourne

Original Version: 1st March 2018

Subject co-ordinators: A/Prof. Suzie Sheehy & Prof. Christian Reichardt

Contents

1 Introduction **2**
1.1 Motivation and Biological Background . . . 2
1.2 Research Methodology . . . 3

2 ECG Raw Data **4**
2.1 Data Processing . . . 4
2.2 Preliminary Signal Exploration . . . 5

3 QRS Complex Identification **7**
3.1 Filtering and Pre-Processing . . . 7
3.2 Basic Algorithm . . . 8
3.3 Algorithm Evaluation . . . 9
3.4 Algorithm Extensions . . . 10

4 Respiratory Rate Estimation **13**
4.1 Spectrum Analysis Methods . . . 13
4.2 RSA vs RPA Performance . . . 15

5 Machine Learning Approach **21**
5.1 Background . . . 21
5.2 QRS Identification . . . 21
5.3 Respiratory Estimation . . . 22

6 Discussion, Conclusion and Reflection **24**
6.1 Discussion . . . 24
6.2 Conclusion . . . 25
6.3 Area for future research . . . 25
6.4 Reflection . . . 26

Bibliography **26**

1 Introduction

This report is submitted as part of the assessment for the authors enrolment in Masters class The Art of Scientific Computation at the University of Melbourne. The report outlines the process the author went through when completing the Respiratory Rate research project.

1.1 Motivation and Biological Background

Respiratory rate, defined as the number of complete breaths per minute, is a fundamental vital sign, with importance across healthcare and exercise science [2],[9]. A persons Respiratory Rate is impacted by a number of factors including injury, illness or stress, adding difficulty to the measurement process, however it is an extremely important vital as abnormal Respiratory Rate is often the first indicator of illness [2]. Despite this, Respiratory Rate monitoring is still not performed routinely in the healthcare setting, which may be due to the lack of standard Respiratory Monitoring technology [9]. One area of critical importance in Respiratory Rate Estimation is finding a low-cost, consistent and robust method for measuring Respiratory Rate, this is most critical in rural, low income areas where the lack of accurate, easy to obtain measurements may led to patients being incorrectly diagnosed, leading to ill patients receiving inappropriate care or no care at all [15], [9]. This issue is pertintent to the diagnosis of Pneumonia, especially in children; Pneumonia is responsible for 14% of all deaths of children under 5, despite pneumonia being easily treatable with widely available oral medication, indicating the importance of being able to effectively diagnose in settings where resources are scarce [19]. Respiratory Rate can be an indicator of severe pneumonia, and the World Health Organisation suggest that Respiratory Rate is integral to the diagnosis pathway; further illustrating the criticality of determining reliable methods for measurement. [9].

An area of active research is developing a method for reliably estimating Respiratory Rate via electrocardiogram (ECG) signals. ECGs measure the electrical activity of the heart, specifically voltage over time, electrodes (typically 10) are placed on a person lying down, these electrodes measure the magnitude of the electric potential from 12 different angles - this is called a 12 lead ECG [18]. ECGS are are a non-invasive, widely available tool used in healthcare, indicating that if a reliable method to measure Respiratory Rate from ECG signals can be developed, that it could be implemented as a diagnosis tool for respiratory illness, especially in low income areas where other healthcare tools aren't readily available.

A normal ECG signal is made up of 3 parts, the P-wave, QRS complex and T wave associated to different activities of the hearts beating cycle [18]. Figure 1.1 illustrates the signal characteristics of each part.

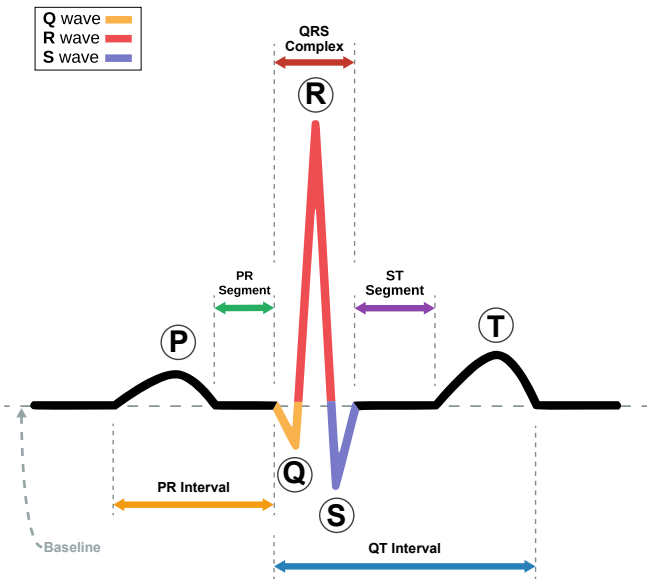


Figure 1.1: Notated diagram of a Sinus Rhythm, the signal generated by an ECG [18].

Of interest to this project is the QRS complex characterised by large short peaks on the ECG signal, corresponding to repolarisation of the ventricles, that is the contraction of the heart to pump out blood [18].

There are two commonly used methods to estimate Respiratory Rate we explore that use data from these peaks (sometimes refered to as RR peaks), providing a potential low-cost widely available methods for measuring Respiratory Rate.

The predominant goal of this project is too progress towards estimating Respiratory Rate from an ECG signal. We investigate methods that are based upon spectral analysis of QRS complex data, specifically the time between QRS complexes and the amplitude of QRS complexes this is explored in section 4. It's first pertinent to develop a robust algorithm for identifying these complexes. Identifying QRS complexes can often present a number of challenges and this project spent a large amount of its time devoted towards this challenge as its the first crucial step in developing a method to reliably estimate Respiratory Rate, we explore this in section 3. As the morphology of an ECG signal can vary based upon both biological and measurement factors, its important to gain an understanding of the varying behaviour to expect so we are able to test the robustness of algorithms against these varied behaviours, this is explored in section 2. We additionally begin exploring a machine learning based model, using a Random Forest Classifier, this is outlined in section 5.

1.2 Research Methodology

The research approach of this project is;

1. Investigate ECG signals, identifying datasets with distinct varying behaviour to test the Robustness of our algorithms. This is described in Section 2.
2. Develop, evaluate and refine an algorithm to robustly identify RR peaks, testing the performance against annotations for the occurence of a QRS complex. This is described in Section 3.
3. Evaluate and test different methods for estimating Respiratory Rate, by analysing their ability to detect a strong, consistent frequency that with high probability is the Respiratory Rate. This is described in Section 4.
4. Begin developing a machine learning model framework for QRS complex identification and subsequent Respiratory Rate estimation. This is described in section 5.

2 ECG Raw Data

The data used in this project is sourced from the Apnea-ECG dataset [11], [13]. There are 75 datasets each corresponding to the sleep of an individual, they contain an ECG signal with amplitude in millivolts, along with annotation files. Every dataset has a QRS annotation file that contains time annotations for the occurence of QRS Complexes, whilst 35 of the files contain a further annotation file indicating the presense of Apnea in minute increments [11]. The data is stored using WFDB conventions.

Figure 2.1 encapsulates the form we want to convert the data and annotations into, i.e an array of waveform values and an associated time array, along with arrays that capture annotation value and time of occurence.

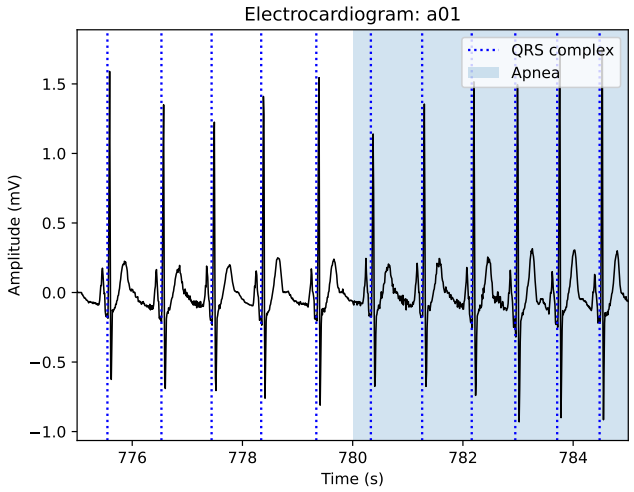


Figure 2.1: ECG signal along with QRS and Apnea Annotations.

2.1 Data Processing

To begin to analyse the ECG signal, we first need to understand how the data is stored. The data for the ECG waveform is stored in binary, and the way to parse the data is described by a header file that is written in ASCII. The description for how to read these header files is provided by [12], under WFDB file formats (header). A header file contains two lines, the record line contains information that is relevant to all signals, for us the important information is regarding sampling frequency and number of signals, allowing us to extract our timing information. The next step is to understand the signal, this is described by the signal line. The key details are the format for which to extract the signal (this specifies how to convert binary into an integer data type), the specific formats are described by [12], under WFDB file formats (signal). In the case of this project, the data is stored in format 16, that is in 16 bits stored least significant byte first (i.e little endian) [11], [12]. This means the integer is given by arranging the bytes as second byte followed by first byte. Additionally, there is information about the ADC (anolog to digital conversion), of relevance to this project is the ADC gain, dividing by this value will scale the integer data into mV ready now for us to plot.

Extracting annotations requires a little more work, and the full description for how to go about this is given by [12], under WFDB file formats (annot). The overview is as follow, the data comes in a pair; an A value and an I value, the A value corresponds to the annotation in our case normal or Apnea occurence and the I value whichs specifies how long after the previous annoation this annotation occured, for the first one that is time after the start for which it occured. The information for these two comes in 16 bits, the 6 most significant bits (in our case the last 6 as its implemented least significant byte first) correspond to the annotation, and the first 10 bits correspond to the interval. There is one caveat, that is if the annotation gives the numerical value 59 (note there are other values but these are irrlevant to this projects data), this specifies a skip and needs to be handled differently, as it means the next 4 bytes of information tell you an extended time interval for which to add to the interval of the upcoming annotation (the next annotation takes longer to occur). The binary information of the next 4 bytes can be thought of as two chunks of bytes, the first being the high and the last the low, each chunk read using the little endian method as we have done before. Then the interval is given by the integer associated with high chunk then low chunk.

Figure 2.2 summarises how each data form is read.

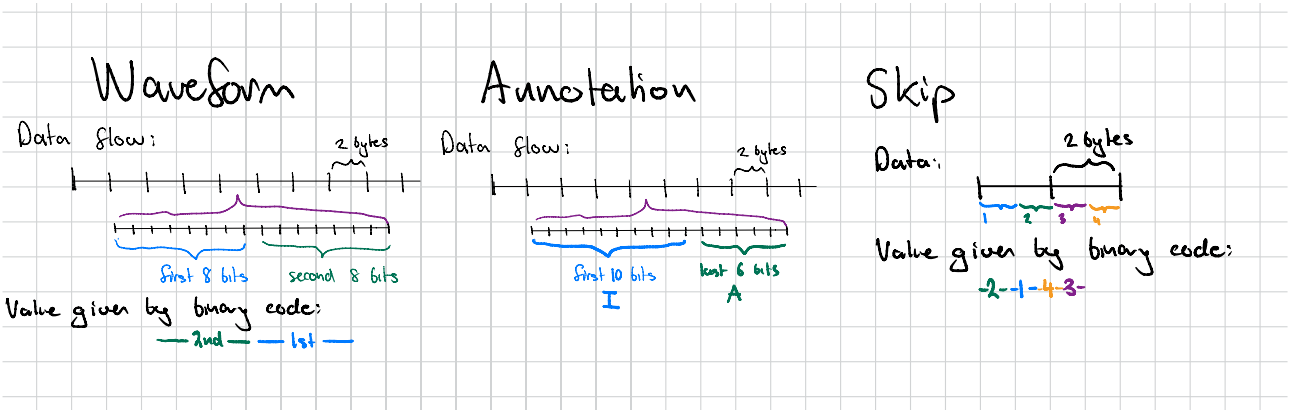


Figure 2.2: Visual for how to process each data form.

2.2 Preliminary Signal Exploration

As we are working with ECG signals, the amplitudes and time intervals differ across time and samples due to morphological differences in humans, along with noise and artifacts picked up in the data collection process [15]. We now illustrate a few samples that show these differences, these samples will give a basis of data to test our algorithms robustness.

Figure 2.3 shows how the amplitude of the QRS complex can vary signficantly across datasets, a12's QRS complexes have very small amplitude whilst a13s has large amplitudes. This can be due to a range of biological facts such as age and gender, as well as measurement factors such as noise [1].

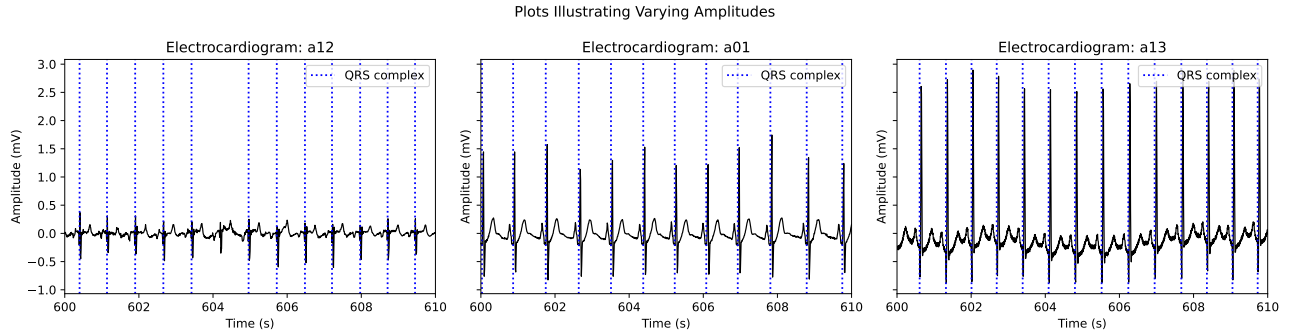


Figure 2.3: ECG signals illustrating differing amplitude magnitudes of QRS complexes across datasets.

Figure 2.4 illustrates that the time between QRS complexes occurence also varies, which along with other factors corresponds to variation in heartrate. In x04 the time between complexes varies between 1.08 to 1.26 secs, in a03 its between 0.93 - 1.00s and in b01 0.68 - 0.74secs ignoring the region of noise.

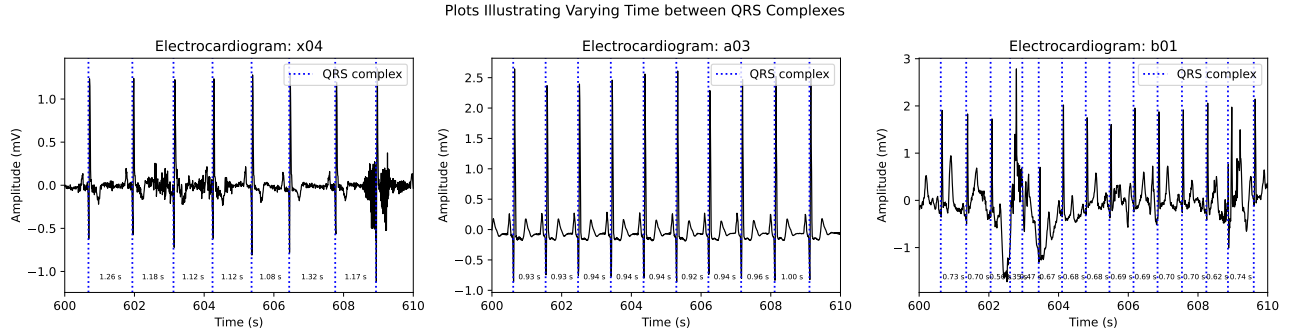


Figure 2.4: ECG signals illustrating differing time intervals between QRS Complexes across datasets.

Figure 2.5 highlights the existence of noise and artifacts in some datasets.

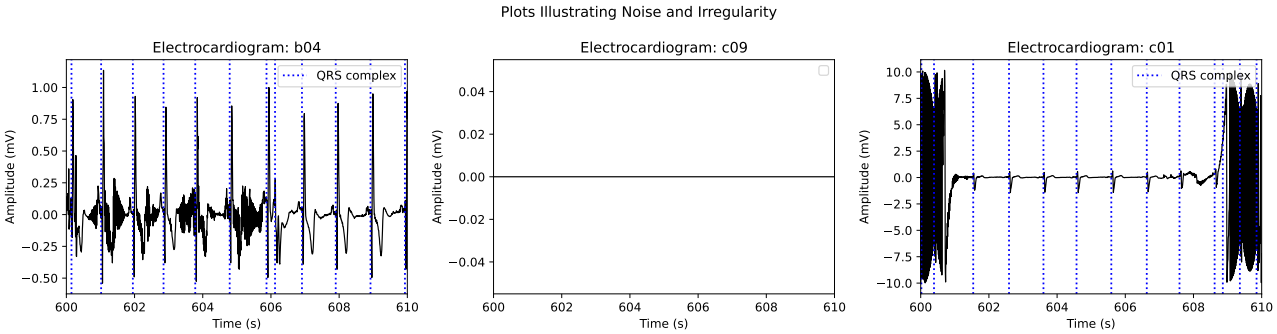


Figure 2.5: ECG datasets illustrating the existence of noise and artifacts in the data.

Figure 2.6 illustrates a particularly important behaviour that within a data set the QRS complex amplitude can be only slightly larger than the amplitude of other peaks (T, P) in the data, this is especially important in section 3 where the amplitudes of QRS complexes play a role in their identification.

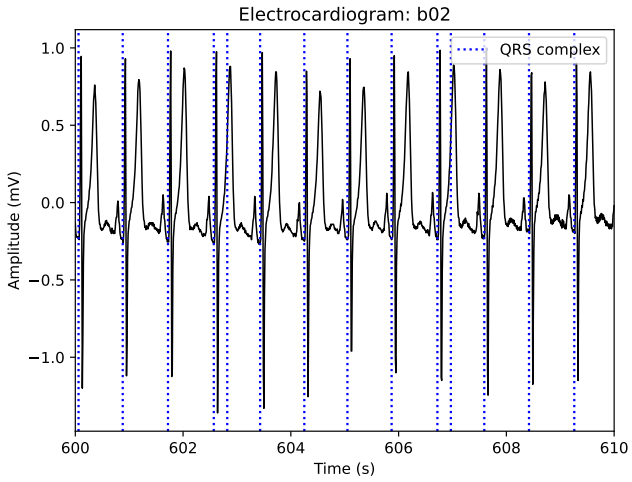


Figure 2.6: ECG signal illustrating QRS amplitude can be close to other peak amplitudes.

Going forward we focus on testing our algorithms against these data samples: a01, a12, a13, a03, x04, b01, b02, c02, as they represent how amplitude of QRS complexes, and the time between can vary. This is important as those characteristics will form the basis of our Respiratory Rate estimation and we need to ensure that the QRS peak identifcation algorithm is robust against variation in these traits.

3 QRS Complex Identification

In this section we develop and evaluate methods for identifying QRS complexes otherwise known as RR peaks in ECG data. The method we develop is inspired by the Pan-Tompkin Algorithm [10]. The Pan-Tompkins algorithm is widely used in literature for ECG analysis, [6], hence why we use it as inspiration in this project. We implement simplifications of the process whose performance we evaluate against the QRS annotations supplied, which indicate the occurence of a QRS complex.

3.1 Filtering and Pre-Processing

ECG signals are prone to noise, and variation in the data, so its important to filter and pre-process before attempting to locate QRS peaks. The filtering is based off literature values for ECG signals, specifically we expect to be locating frequencies associated within the range of 5 - 15 Hz [10]. Figure 3.1 illustrates an example waveform before and after filtering in different frequency ranges, employing a Fourier Transform before removing frequencies and inverting back.

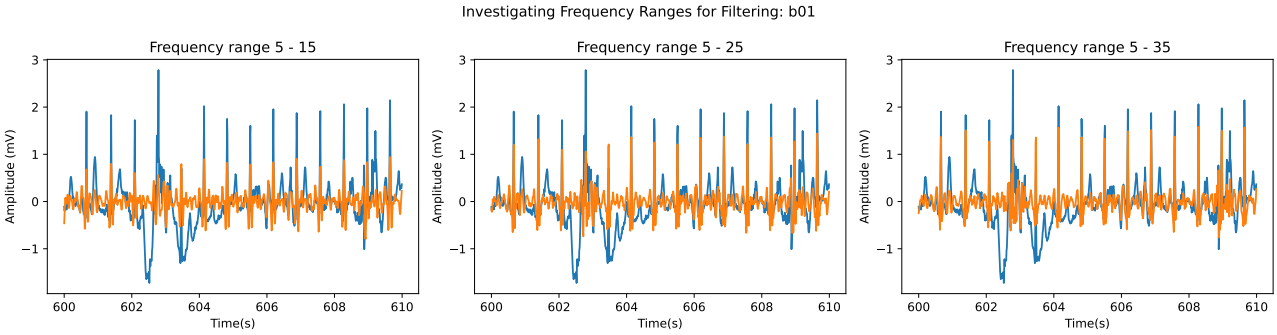


Figure 3.1: Frequency Filters applied in different ranges, blue indicating the raw data and orange the filtered.

We can see that a frequency filter cleans up the waveform to better resemble a ECG signal, whilst retaining the QRS complexes, Figure 3.1 suggests that a range of 5-35 Hz mantains amplitude of the QRS complex along with the ECG signal shape, this is the frequency range used in this project.

Once we restrict ourselves to a filtered waveform, we implement a pre-processing method heavily influenced by [10]. The idea is that QRS peaks are associated with large inclines and declines in the wave form, so we want to use derivatives, and square them to highlight and exhausterbate this difference, before a moving integration window converts the derivative back into a smooth signal with a large peak associated with the QRS peaks whilst minimising the effect of the other smaller peaks. The process is specifically,

1. Differentiate the waveform.
2. Square the differentiated waveform, to ensure all values are positive, additionally the steeper incline and decline , associated with the peaks will now be amplified helping to distinguish against the smaller peaks.
3. A moving integration window on the squared differentiated waveform, produces a signal that amplifies the QRS peak (nearby it), and minimises the other smaller peaks.

Figure 3.2 illustrates how the waveform is transformed over this process, note in the overlap the processed waveform has been scaled to highlight how the peaks are close to each other.

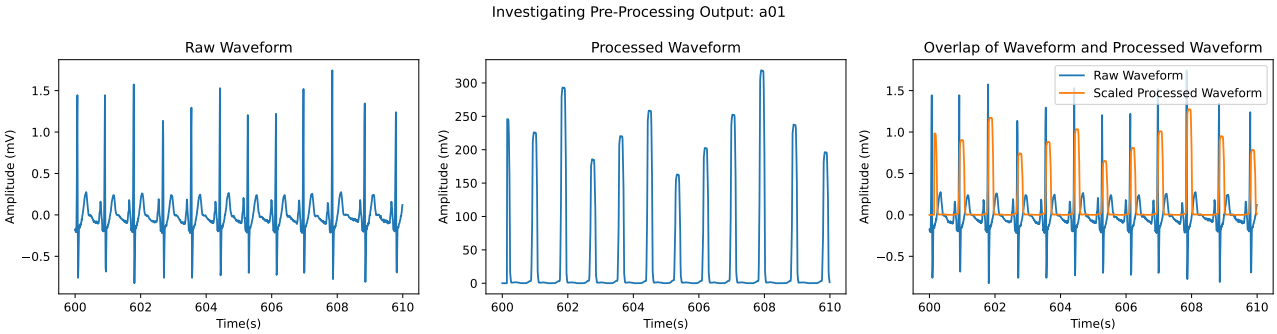


Figure 3.2: Illustration of the Pre-Processed Waveform

3.2 Basic Algorithm

We can see in Figure 3.2 that the processed waveform has large peaks close to (but just after) the location of the QRS peaks in the Signal. The idea of our algorithm is to,

1. Locate the peaks on the processed waveform which are clear and can be done using an inbuilt python peaking finding function (sp.signal.find_peaks).

2. In a small search window around the location of these peaks, move back onto the original waveform and in this interval identify the largest value and the time associated with it, this is what the algorithm identifies as a peak.

The search window is chosen as 0.25 secs, this is not too large as the normal heart rate is between 60-100 bpm corresponding to 0.6 - 1sec between QRS time complexes [17]. When implementing the inbuilt function, we need to prescribe a size threshold for what we expect a peak in the integrated waveform to be above. As we have seen that amplitudes can vary significantly, the idea of our algorithms is to then start with a high size threshold value, we choose 1000 as the dataset with the largest amplitudes we identified (a13) was roughly 700 (note the scaling is 1/200 in Figure 3.2), so 1000 was chosen as a buffer. Starting from the high size threshold we see how many peaks are identified, if not enough peaks are identified with respect to some time interval threshold (i.e number of peaks per sec) we gradually decrease the size threshold until the number of peaks identified is above the minimum number we expect to identify. This is the basic idea of the algorithms developed in this section, Figure 3.3 summaries the process,

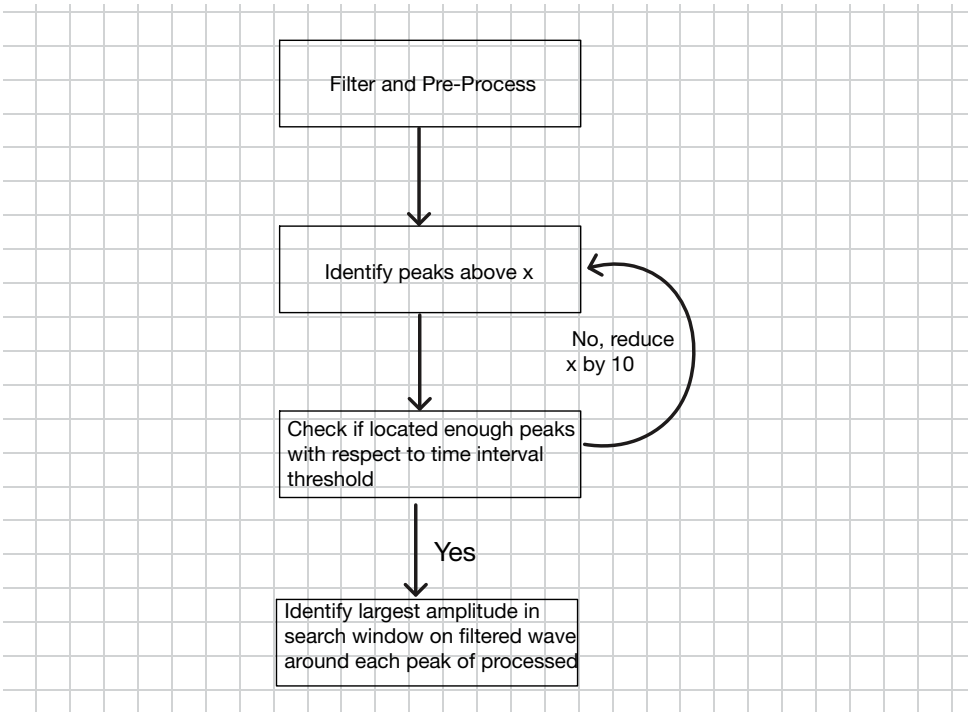


Figure 3.3: Flowchart illustrating basic algorithm process.

Figure 3.4 provides a visual description, the orange crosses indicate the peaks located on the Processed Signal, the blue windows illustrating the search window for which we look to locate peaks on the ECG signal, with the red crosses the peaks located.

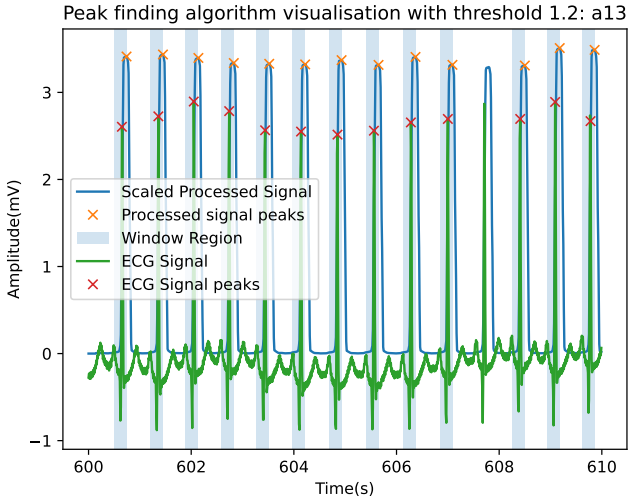


Figure 3.4: Plot illustrating the basic idea of the peak finding algorithms developed in this paper

The pseudocode below summarises the basis idea for implementing the base algorithm.

Algorithm 1 Basic Algorithm form

```
filter_signal ← Filter(signal)
processed_signal ← Preprocessing(filter_signal)
threshold ← 1000
length_peaks ← 0
while length_peaks < (end_time − start_time)time_interval_threshold do
    peaks ← find_peaks(processed_signal, threshold)
    length_peaks ← length(peaks)
    threshold = threshold −10
end while
for i = 1, . . . , length(peaks) do
    region ← filter_signal[search_range(i)]
    peaks_indices[i] ← argmax(region)
end for
```

3.3 Algorithm Evaluation

In Figure 3.4 we can see that with a time interval threshold of 1.2 peaks per second that a peak has been missed. To test which threshold is most effective, we test the number of QRS complexes each algorithm identifies against the number of QRS annotations as a preliminary evaluation metric, this is summarised by Table 3.1.

Table 3.1: Proportion of peaks detected for different threshold values between 0.8 and 1.4 in the first hour of data.

Threshold	0.8	0.9	1.0	1.1	1.2	1.3	1.4
a01	0.74	0.87	0.92	1.01	3.14	3.14	3.14
a12	0.72	0.88	0.88	0.98	3.04	3.04	3.04
a13	0.6	0.67	0.76	0.82	0.91	0.97	2.76
a03	0.83	0.92	0.99	3.51	3.51	3.51	3.51
x04	1.05	1.19	5.17	5.17	5.17	5.17	5.17
b01	0.75	0.86	0.96	1.0	3.16	3.16	3.16
b02	0.74	0.74	0.86	0.9	2.7	2.7	2.7
c02	0.89	0.96	1.06	3.85	3.85	3.85	3.85
Mean Detected	0.79	0.88	1.45	2.15	3.18	3.19	3.42

We can see that different time interval ratios are better than others, namely between 0.90 and 1.0, investigating between 0.90 to 1.0, we have:

Table 3.2: Proportion of peaks detected for different threshold values between 0.9 and 1.0 in the first hour of data.

Threshold	0.9	0.92	0.94	0.96	0.98	1.0
Mean Detected	0.88	1.39	1.41	1.44	1.44	1.45

So the optimal number of QRS peaks per second threshold value seems to be 0.90. We do a more in-depth analysis into 0.90, noting that above only compared number of peaks identified, not locations. Table 3.3 investigates locations, specifically the proportion of matching peaks, along with the peaks hallucinated and peaks missed.

Table 3.3: In depth analysis into 0.90 threshold, proportion of peaks found, peaks hallucinated and peaks missed

Files	Matching	Hallucinated	Missed
a01	0.866	0.0	527.0
a12	0.871	34.0	542.0
a13	0.671	3.0	1600.0
a03	0.914	9.0	314.0
x04	0.992	517.0	21.0
b01	0.851	27.0	590.0
b02	0.735	18.0	1193.0
c02	0.937	68.0	216.0

Its clear that the algorithm isn't perfect, especially looking at x05 it can hallucinate i.e identifies peaks that aren't peak, and looking at the missed column it still misses a lot of peaks as well.

3.4 Algorithm Extensions

The problem with a fixed time interval threshold is the variability in the time between QRS complexes across datasets. The following extensions of the algorithm described above attempt to get around this problem in different ways.

One way we could is by noticing that a large jump often occurs from values that are close to the estimated correct number of peaks to a number far higher, intuitively this is because we are now factoring in other peaks (T, P for example). The first extension algorithm detects this jump, and then chooses the peaks identified from the time interval threshold before that jump. When detecting this jump we test time interval thresholds between 0.5 to 1.6 as the results indicate these capture all jumps we expect. Table 3.4 summarises the performance of this algorithm.

Table 3.4: Performance of the jump based algorithm.

Files	Matching	Hallucinated	Missed
a01	1.0	27.0	1.0
a12	0.969	53.0	130.0
a13	0.967	5.0	158.0
a03	0.986	9.0	52.0
x04	0.992	538.0	21.0
b01	0.993	45.0	28.0
b02	0.894	21.0	479.0
c02	0.989	231.0	39.0

Its clear from this table that this algorithm performs far better than any of the fixed time interval threshold based algorithms, with the proportion of matching QRS complexes located being considerably closer to 1 in every dataset, outside of b02 the proportion ranges from 0.967 - 1.0. However, it still isn't perfect with still large amount of hallucination in x04, whilst it still misses considerably many peaks in some datasets such as b02. We saw in section 2, that the b02 sample has very small difference in QRS complex amplitude compared to other

peaks, indicating the sensitivity of this kind of algorithm to relative magnitude differences of QRS complex amplitude and T, P peak amplitudes. The sacrifice with this algorithm is that the increased complexity reduces the efficiency of the algorithm. Table 3.5 illustrates that the run time over an hour of data increases up to about 15 times.

Table 3.5: Time Comparison (sec) for running each algorithm over an hour of data

	a01	a12	a13	a03	x04	b01	b02	c02
Jump Algo	3.58	3.35	1.27	2.53	3.48	2.62	2.8	2.84
Basic Algo with Threshold 0.9	0.32	0.4	0.17	0.26	0.43	0.31	0.32	0.28

In the basic algorithm the while loop gets replaced with the following, its clear how the time complexity increases.

Algorithm 2 Jump Based Modification to location of Processed Signals Peaks

```
threshold ← 1000
initialise arrays to store peaks (peaks_list) and number of peaks identified (peaks_size)
for i = 5, ..., 16 do
    time_interval_threshold ← i × 0.1
    length_peaks ← 0
    current_threshold ← threshold
    while length_peaks < (end_time − start_time)time_interval_threshold do
        peaks ← find_peaks(processed_signal, threshold)
        length_peaks ← length(peaks)
        current_threshold = current_threshold −10
    end while
    peaks_list[i − 5] ← peaks
    peaks_size[i − 5] ← length(peaks)
end for
found ← False
for j = 1, ..., length(peaks_size) − 1 do
    if peaks_size[j + 1] / peaks_size[j] ≥ 2 then
        peaks = peaks_list[j]
        found ← True
        break
    end if
end for
if found = False then peaks = peaks_list[−1]
end if
```

Another improvement that is only relevant to the data sets used in this project, is by basing the time interval threshold as a factor of the number of QRS annotations, this is clearly not a very viable algorithm however as it can't be used on other datasets but is worthwhile evaluating as is if its robust on these datasets it may be a good candidate when we test Respiratory Rate estimation methods. Tables 3.6 summarise the performance of this algorithm.

Table 3.6: Performance of the annotation based algorithm with factor 0.90 and 0.95

Factor	0.9	0.9	0.9	0.95	0.95	0.95
File	Matching	Hallucinated	Missed	Matching	Hallucinated	Missed
a01	0.92	0.0	310.0	0.97	0.0	121.0
a12	0.97	51.0	130.0	0.97	51.0	130.0
a13	0.91	5.0	453.0	0.95	5.0	221.0
a03	0.91	9.0	314.0	0.96	9.0	153.0
x04	0.87	84.0	355.0	0.93	92.0	193.0
b01	0.95	28.0	180.0	0.95	28.0	180.0
b02	0.9	21.0	441.0	1.0	7648.0	12.0
c02	0.92	68.0	273.0	0.94	68.0	216.0

We can see that this algorithm definitely has potential, especially with a factor of 0.90 avoiding large hallucinations, however it misses many peaks in each dataset, with a factor of 0.95 doing similarily except for b02 which has significant hallucination. Here the algorithm is almost identical to the basic algorithm form with the modification that:

$$\text{time interval threshold} = \text{factor} \times \frac{\text{number of annotations}}{\text{end time} - \text{start time}}.$$

4 Respiratory Rate Estimation

When estimating Respiratory Rate from ECG signal, the common approach is to use data associated with the QRS complexes [8]. There are two common methods; the first focuses on locating the dominate frequency in the time between QRS peaks, this corresponds to the biological phenomenon of Respiratory Sinus Arrhythmia (RSA), which states that heart rate speeds up when you breath in and slows down when you breath out [5], [8]. The data we are working with here is 2D, with each pair corresponding to the length of time between QRS peaks and the midpoint of that time interval. The other method is based on the amplitude differences of the QRS peaks (RPA), which is associated to impacts breathing in and out has on the electrical output of the heart [8]. The data we are working with here is similarily 2D with each pair corresponding to the amplitude of Peak and the time at which it occured. The idea behind both methods is to then isolate the frequency with the strongest associated amplitude, where the assumption is that this frequency corresponds to the Respiratory Rate whilst the less dominate frequencies correspond to noise and other biological processes and artifacts. The normal Respirate Rate range is 12 - 20 BPM [4], which corresponds to a range of 0.2- 0.33Hz , for a candidate Respiratory Rate. Note that we can't ignore frequencies outside of the range especially as we know we are dealing with data where Apnea and possible other Respiratory events are occuring that cause Respiratory Rate to land outside of the normal range - hence we a predominantly looking for a consistent dominate frequency within a range of 0.05 - 0.50 Hz (to avoid noise). In all of the work below, the QRS complexes are identified using the Jump Detection Algorithm described above.

4.1 Spectrum Analysis Methods

Implementing either of these methods requires spectrum analysis, however as we are dealing with non-uniform time data we can't blindly apply the Fast Fourier Transform (FFT). There are two ways of dealing with non-uniform time series data which we explore in this project. The first is to use an interpolation method between the QRS peak data and then sample a uniform time from this and apply the FFT to this waveform, the other is to use the Lomb-Scargle Method defined for non-uniform time series data. We give a brief explanation of each method, and compare the output spectrum of those in this section, then the section below we will continue the analysis to the spectrograms.

4.1 Interpolation and FFT method

The idea behind this method is to create an interpolated waveform, specifically by fitting a piecewise cubic function to the data and then sampling uniformly from this wave, this then means we have uniform time and are able to apply a discrete Fast Fourier Transform algorithm. As the time intervals doen't vary largely, as breathing is typical consistent, there is validity to this method. Ideally, we would want to be able to produce a quantitative measure for the accuracy of the interpolation, however methods such as mean square error would basically be zero as an interpolation method ensures that it goes through each data point, thus we settle for visual comparison. In Figure 4.1, 4.2 its clear that the interpolation matches the waveform closely.

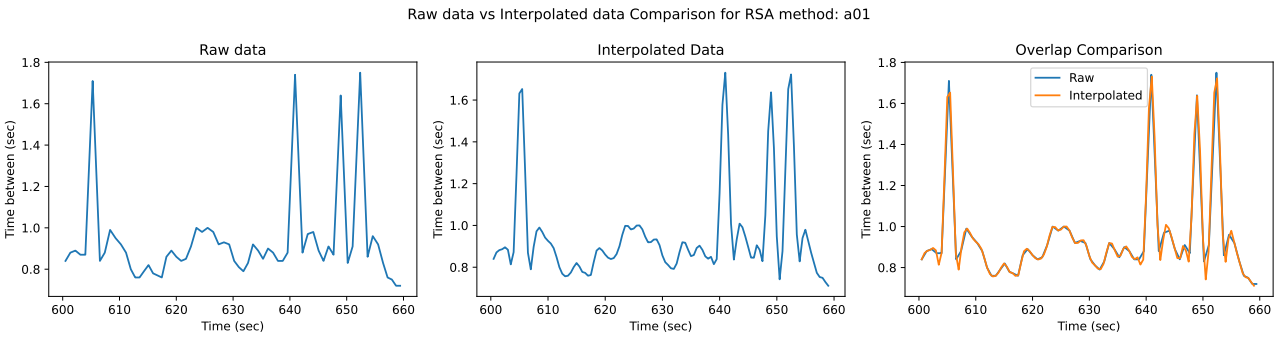


Figure 4.1: Raw data vs Interpolation comparison for RSA method

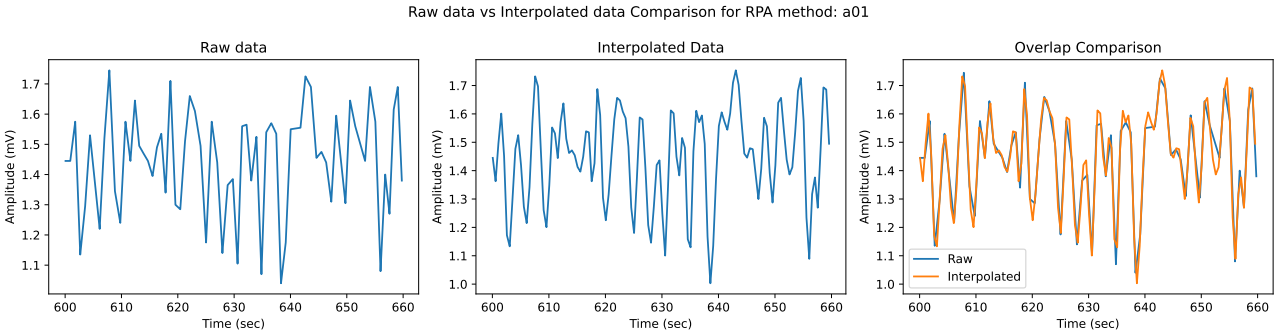


Figure 4.2: Raw data vs Interpolation comparison for RPA method

The discrete Fourier Transform (FFT) is a method that decomposes a sequence of points into its frequency components. Per the Nquist Theorem, we consider frequencies that are multiples of 1/sample rate. For a sample $(x_n), 1 \leq n \leq N$, where we have N data points.The Discrete Fourier Transform for frequency index k, is given by:

$$X_k = \sum_{n=1}^{N} x_n \cdot e^{\frac{-2\pi i k n}{N}}.$$

Note that X_k is a complex number, where the magnitude corresponds to the amplitude, and the angle the phase. When plotting spectrums we are looking at the $|X_k|$ against its frequency k. When we refer to a Fast Fourier Transform this refers to an algorithm that computes the transform in the fastest known time complexity ($O(N \log N)$).

4.1 LombScargle Method

The reference, [16], provides a full description and discussion on the LombScargle Method, the method we implement is per SciPy, whose documentation is given by [14], the method SciPy implements is described by [20]. The basic idea is that the LombScargle Method is statistical, where each frequency θ is fitted to a function of the form,

$$f_\theta(x) = a * \sin(\theta x) + b * \cos(\theta x) + c.$$

The fitting is done via weighted least squares estimates. This method has no reliance on uniform time, and thus allows us to directly analyse the spectrum of non-uniform data without relying on first interpolating. Something worth considering when implementing this method is that as it is statistical there is a presence of error in the estimation.

4.1 Method Comparison

In Figures 4.3, 4.4 we can see a comparison of both spectrum analysis methods being applied to each of the RSA and RPA waveforms. The overlap plots show that clearly both analysis methods pick up on the same frequencies, however they do so with different amplitudes, in the RPA case the LomgScargle Method picks up on large frequencies, whilst in the RSA case LombScargle and FFT methods are similiar, with the FFT method amplitudes being slightly larger. These figures indicate that both methods are viable, as they pick up similiar frequency peaks with similiar amplitude proportions (i.e amplitudes of frequency in relation to others within the same method). As a preliminary spectrum analysis doesn't conclusively point towards one of the spectrum methods, we evaluate the Respiratory Rate Method employing both methods.

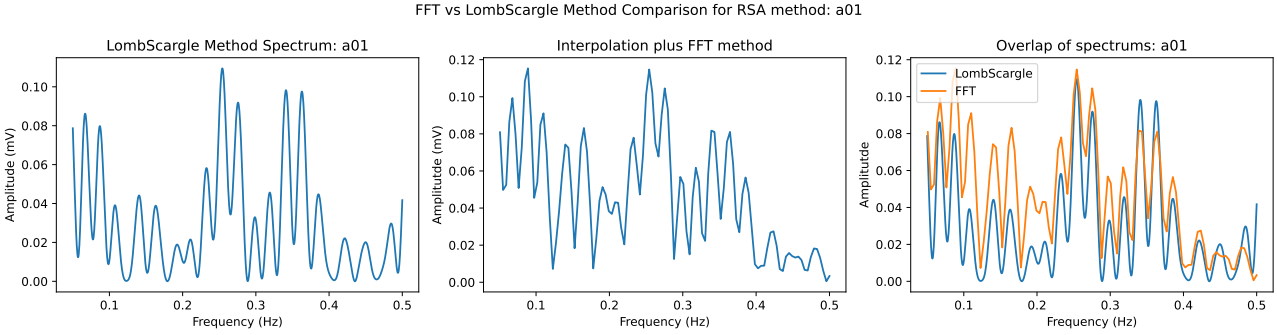


Figure 4.3: Spectrum comparison of LombScargle vs FFT based methods for RSA

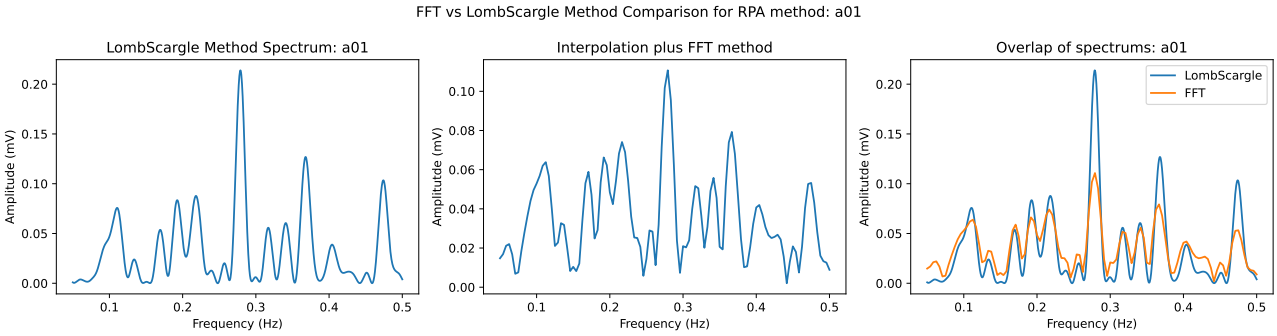


Figure 4.4: Spectrum comparison of LombScargle vs FFT based methods for RPA

In the following section we are going to produce Spectrograms, in producing these plots the time complexity of both methods becomes important, the Fast Fourier implementation is $O(N \log N)$, whilst the LombScargle method is $O(NM)$, where M is the number of frequencies, N the number of data points. This difference becomes significant, and is why for compilation reasoning we focus largely on the FFT based method.

4.2 RSA vs RPA Performance

We begin evaluating RSA vs RPA methods by looking at spectrums, in Figure 4.5, we can see in the overlap plots that both RSA and RPA methods, employing either the FFT or LombScargle spectrum analysis pick up on similiar frequencies. However the a key thing to notice is in our expected Respiratory Rate interval (0.2 -0.33), RSA picks up on two frequency peaks whilst RPA only picks up on one, this is an indication that the RPA method may be able to isolate a dominate frequency better.

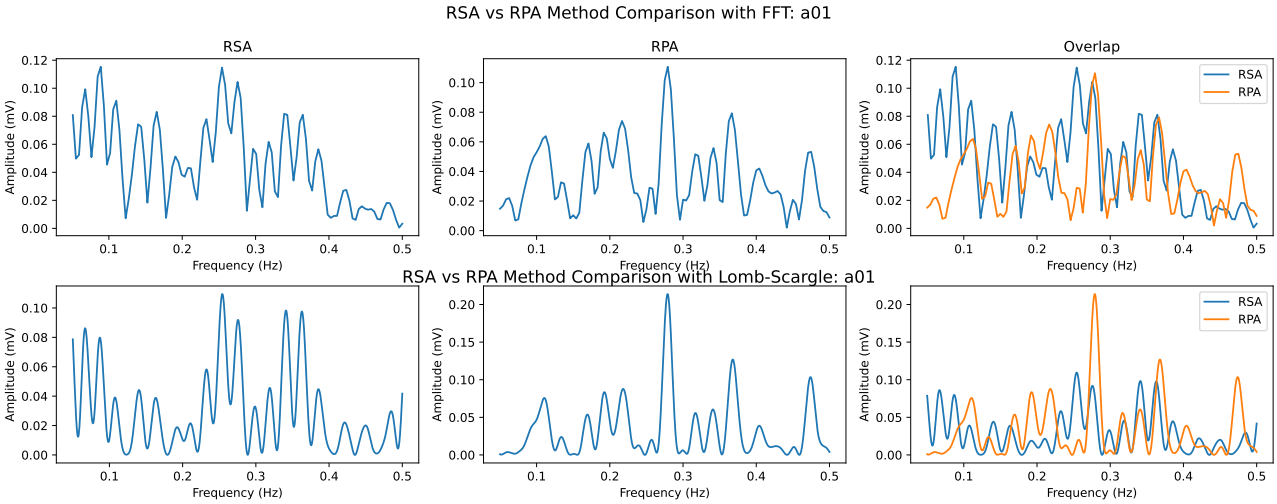


Figure 4.5: Spectrum comparison of RSA vs RPA method

To better evaluate each methods we now focus on analysing Spectrograms, which tell us how the spectrum changes over time, as we expect Respiratory Rate to remain relatively stable [3], we use spectograms to see if we can pick up on a consistent dominate frequency within the expected range. If we are able to, that would be a good indication of the methods ability to estimate the Respiratory Rate. In what follows the spectrograms work on a 60sec window, with 10 sec increments, i.e produces a spectrum every 10secs, based on a 60sec window of data (30 secs each side). In Figures 4.6, 4.7 we can see a Spectrogram over the first hour for each method, implemented with each analysis technique. Its clear in both figures that the RPA method picks up on a stronger dominate frequency sitting just below 0.3Hz compared to the RSA method, which picks up a similiar frequency however with a weaker amplitude.

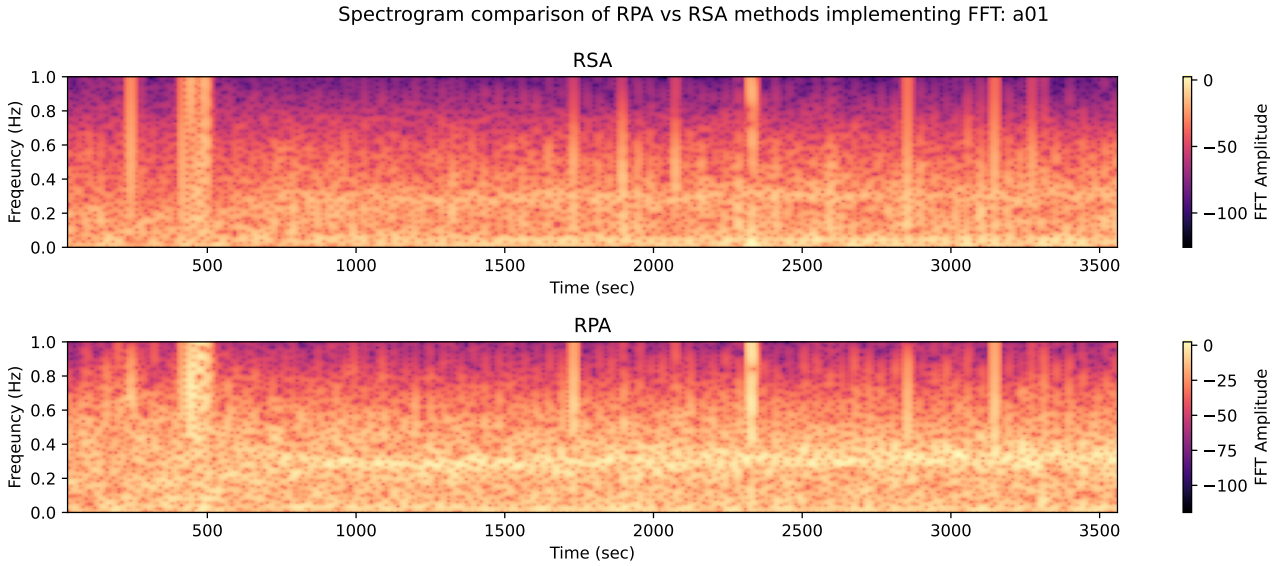


Figure 4.6: Spectrogram comparison of RSA vs RPA method with FFT implementation

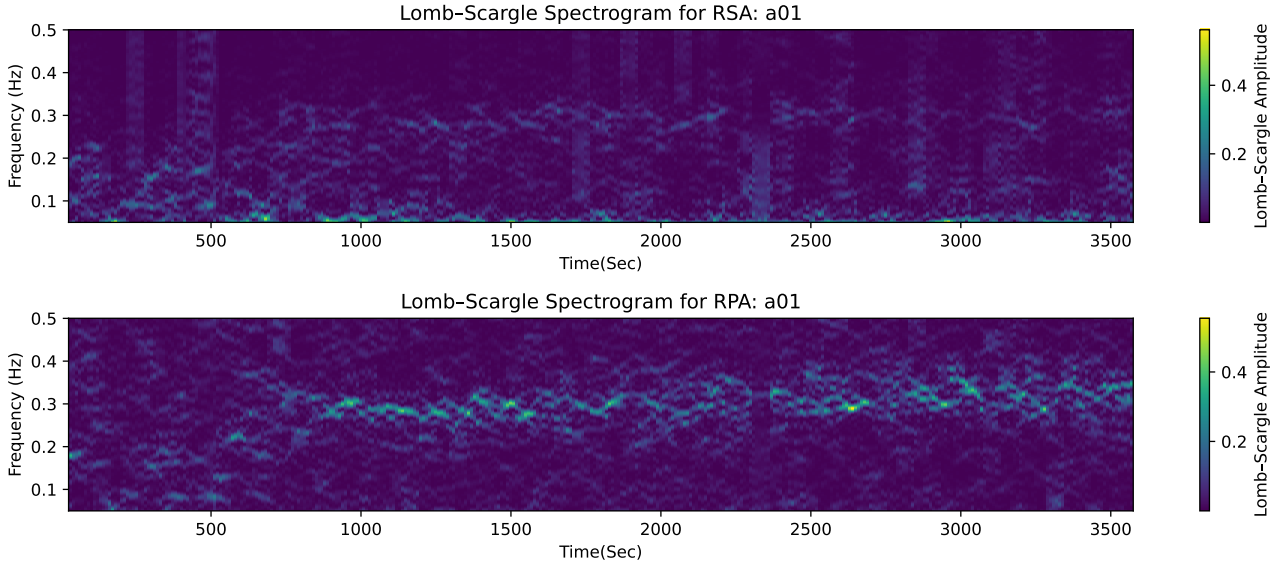


Figure 4.7: Spectrogram comparison of RSA vs RPA method with LombScargle implementation

Thus far it appears that the RPA method seems to perform better than the RSA based method. We now explore the performance of the RPA method, with the FFT implementation as that had the clearest consistent dominate frequency in the Spectrograms above, whilst also being far less computationally expensive. In Figure 4.8 we can see the Spectrogram of the RPA method applied to our 8 test datasets. In the Figure its clear the method lacks robustness as it is only able to detect a dominate frequency pattern in the a01 data set, in the other 7 no clear pattern is visually distiniguishable. This is indicative of the sensitivity of the RPA method to noise as well as incomplete QRS complex data that is present. This makes sense as the method is based upon amplitudes, which would be highly sensitive to artifacts and noise affecting the ability of the method to pick up on the frequency associated with Respiratory Rate, as well as if a peak is hallucinated the amplitude may be dramatically different inpacting the ability to detect a pattern. Note also that in Figure 4.7, the Spectrogram is pixelated and not as clear as the FFT Spectrograms.

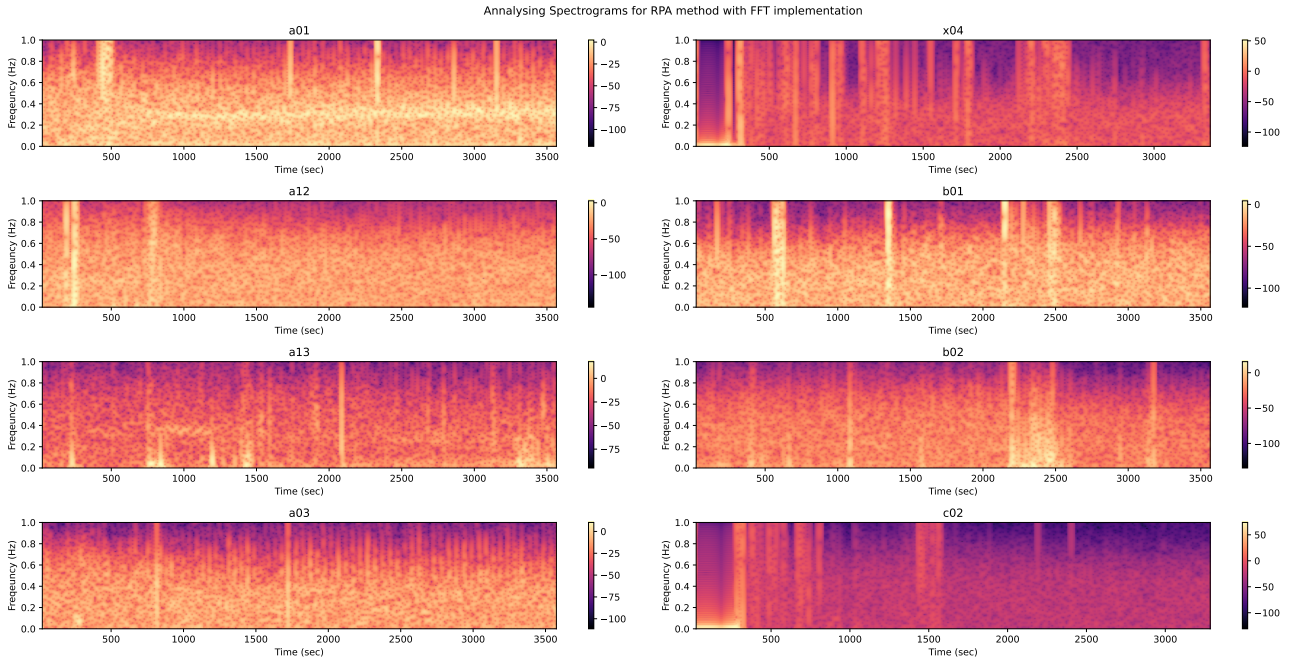


Figure 4.8: Spectrogram comparison across datasets for RPA method with FFT implementation

It is possible that the occurence of QRS complexes is less sensitive to noise, so despite performing worse in the test case we now explore its performance, with FFT implementation, as shown in Figure 4.9.

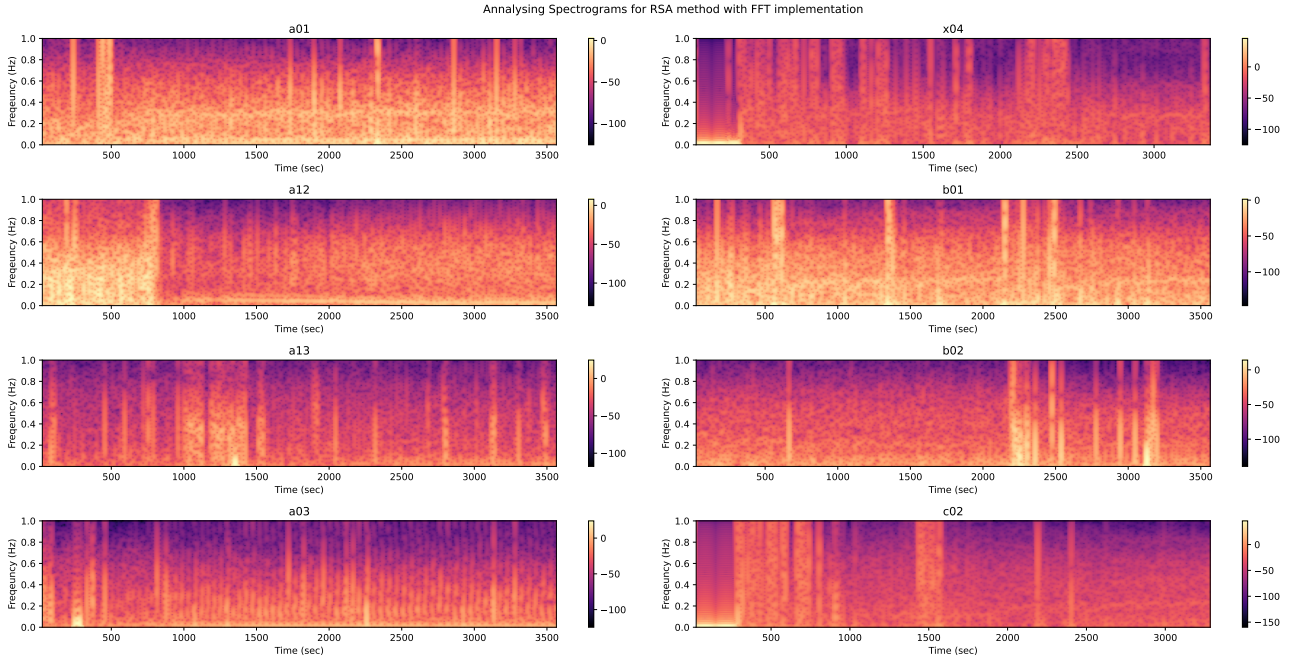


Figure 4.9: Spectrogram comparison across datasets for RSA method with FFT implementation

Looking at datasets b01, c02, b02 there are faint signs of a consistent frequency being picked up, most notably in dataset b01. We note that these datasets had the the largest proportion of peaks correctly identified, indicating the importance of a reliable QRS compelx identification. These faint signals weren't picked up by the RPA method, indicating that whilst the RPA method may be better at picking up a stronger signal, it isn't as robust as the RSA method may be.
In the figures above we have only compared the FFT based implementations, the reasoning is that as discussed the LombScargle method is more computationally expensive, so producing and saving the Spectrograms is time and memory consuming along with that the Spectrograms lack clarity, further the LombScargle method being statistical introduces error into the estimation. We saw that the interpolation method is very similiar in behaviour to the raw data providing validity for working with this method.

Continuing with the RSA method employed with the FFT implementation we explore the best increment and window size, to see if this may improve clarity of the spectrograms. Intuitively, we may expect that increasing the window size may increase the accuracy as there is more data to work with which may reduce the impact of hallucinations or missed peaks, so we begin by exploring a larger window size.

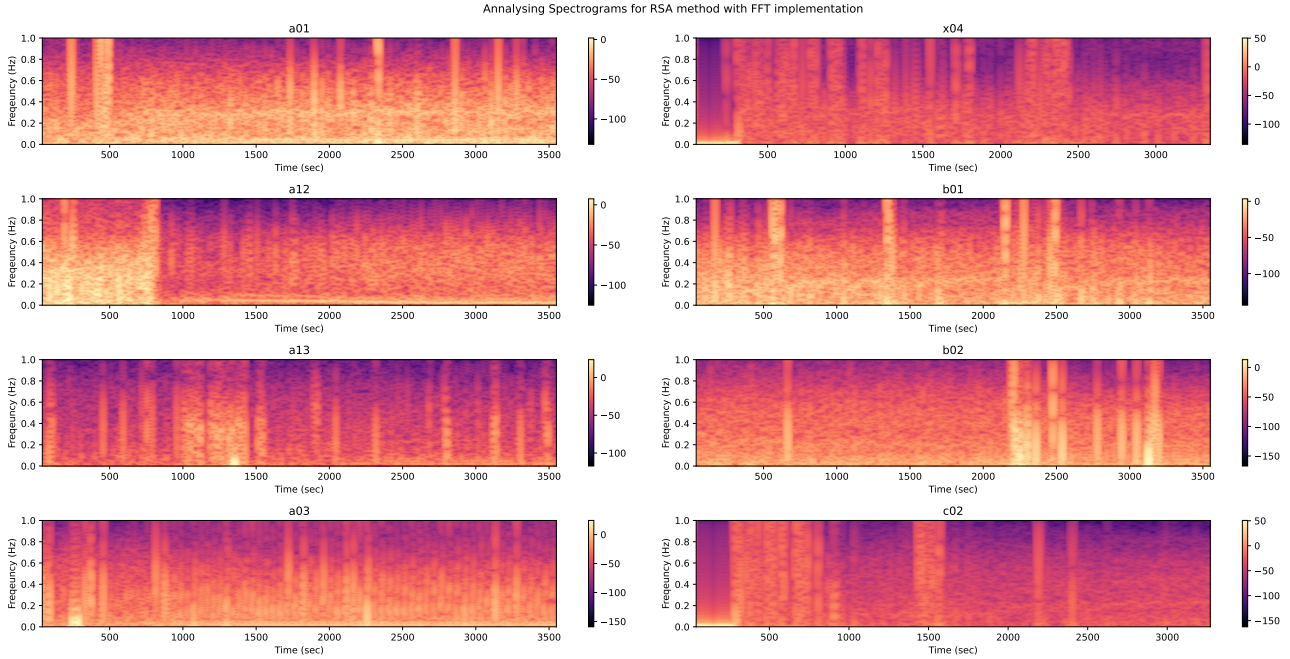


Figure 4.10: Spectrogram comparison across datasets for RSA method with FFT implementation with 10 second increments and 90 second window.

In Figure 4.10 which uses a 90 second window, there is no noticable difference to Figure 4.8 which employs a 60 second window, indicating a 60 second window is sufficient. We now explore increasing the increment size to 30 seconds. In Figure 4.11 it shows the increase to 30 second doesn't appear to change the spectrogram behaviour.

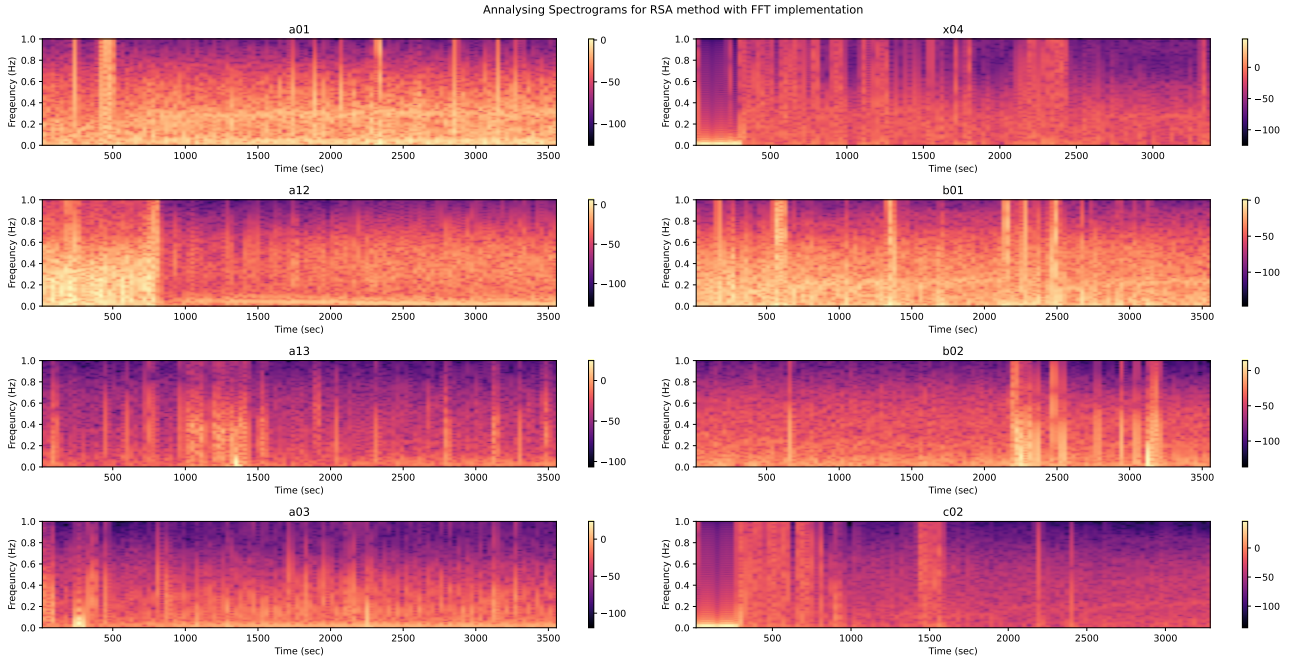


Figure 4.11: Spectrogram comparison across datasets for RSA method with FFT implementation with 30 second increments and a 60 second window.

Initial exploration indicates that increment size and window size aren't contributing factors to the lack of signal in the spectrograms, this suggests that the problem is with either the method or waveforms we are work-

ing with.

It is also worth considering that the QRS identification algorithm may not be the issue. To test this, we use peaks detected by the annotation files instead of our own Algorithm. In Figure 4.12 we can see that using the annotation files, the spectrogram looks very similiar to that produced by the Jump Algorithm seen in Figure 4.9, suggesting that the either there are errors in the annotations or that the RSA method may not be able to always pick up on frequency trends. Note this is also an indication that the RSA method may be robust to missing or incorrect peaks as there is variance in the Jump Algorithm and QRS annotation files, as seen in section 3.

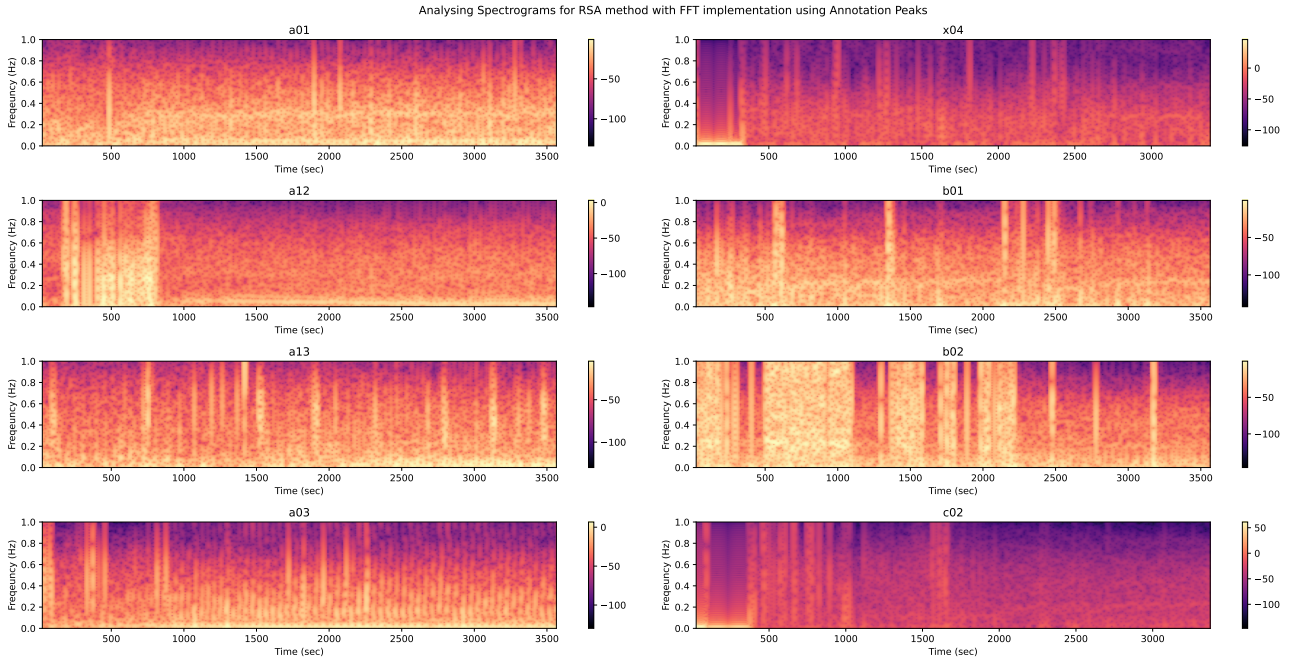


Figure 4.12: Spectrogram comparison across datasets for RSA method with FFT implementation with peaks identified via annotations.

In Figure 4.13 we can see that using the annotation files, the RPA method is unable to detect any trends, compared to the RPA method utilising the Jump Algorithm which did detect a trend as seen in Figure 4.8. This doesn't have a clear explanation, considering the precision of the Jump Algorithm compared to annotation files for the a01 dataset. This may suggest that trend is an artifact.

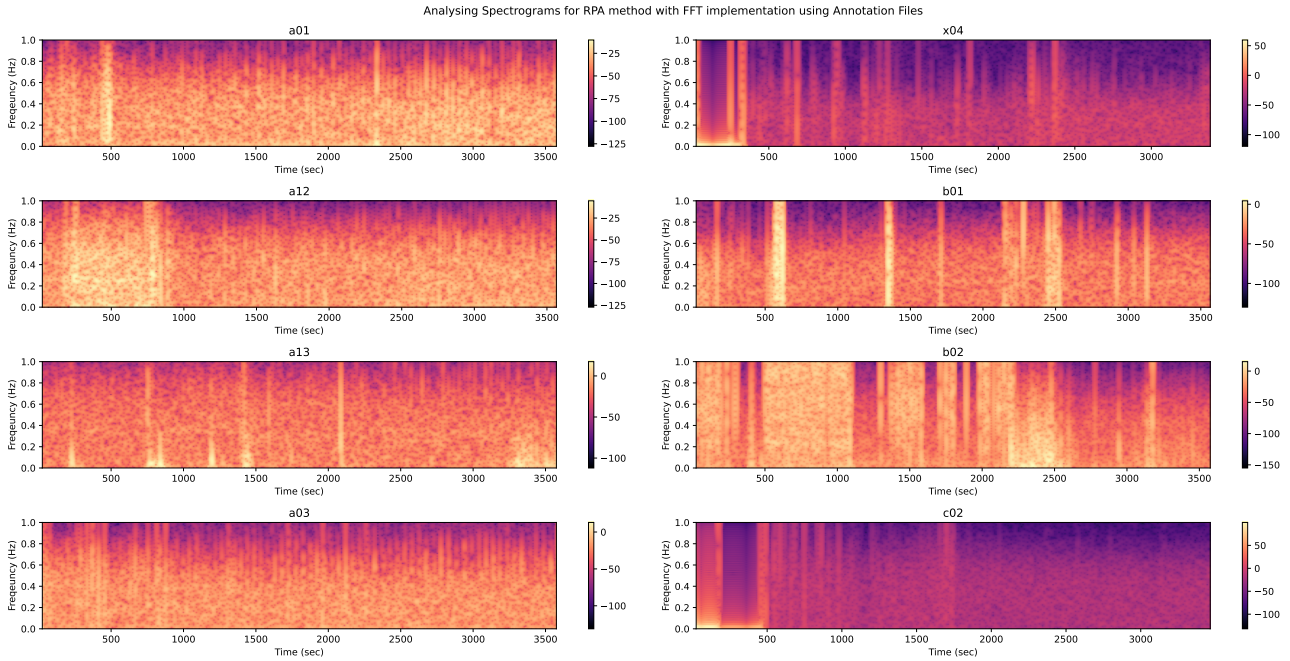


Figure 4.13: Spectrogram comparison across datasets for RPA method with FFT implementation with peaks identified via annotations.

5 Machine Learning Approach

5.1 Background

A random forest is a type of supervised classification or regression model, it is comprised of multiple decision trees that are combined to produce a single result, in this project we use it to classify sample points of an ECG Signal as either an QRS complex or not [7]. Decision trees can be thought of as a tree like flowchart, where you evaluate yes or no conditions at each node, where a yes or no correspond to a seperate branch leading to a new condition to evaluate, or a final result. The idea of decision trees is to seek the best split of the data, against there trained classification data [7]. A random forest, trains each of these trees utilising a bootstrap aggregation method, where multiple training subsets a created with replacement, and each decision tree is trained independently on these subsets and with feature randomness, that is on a random subset of features, to ensure low correlation of the trees (i.e each tree isn't doing the same thing) [7]. The final result of the Random forest is chosen as the majority value (i.e yes or no).

5.2 QRS Identification

We train the model on the first 15 mins of data associated with the datasets: a01, a12, a13, x04, a03, b01, b02. The reasoning for choosing these sets is in section 2 we saw they capture the varying morphology of ECG signals. The data features inputed into the models are the ECG data, derivative, derivative squared, second derivative, Processed Signal (from section 4), and Frequency filtered (5 - 35Hz) data. The motivation behind choosing these features is that they should change for QRS peaks, which can be thought of as large gradient changes, whilst the Process and Filtered Signal help ensure the model is robust against the presense of noise. The training labels for these sets are created from the annotation files, where each data sample recieves as 0 value, unless it is located at the annotation value or 2 samples either side, to provide the model with some cushioning to capture the changing behaviour associated with a peak. The model is built using Scipys RandomForestClassifer, chosen with the following hyperparameters, 100 estimators (i.e decision tress), max tree depth of 30 and with balanced class weight.

To evaluate the performance of the model we test it against ECG data from the 60-65th minute of the same dataset. We first test its performance against the labelled data associated with annotation files using the Precision, Recall and F1 metrics, which are defined as follows. Let TP denote true positives i.e how many predicted (1s) are actually (1s), let FP denote false positives i.e how many predicts (1s) aren't acutally (1s) and let FN denote the number of 1s that weren't detected.
Precision is the porportion of predicted 1s that are acutally 1s, i.e how many samples that are predicted as part of a QRS complex, actually are part of a QRS compelex.

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Recall is the proportion of 1s that are detected, i.e the proportion of samples that are part of the QRS complexes, that have been detected.

$$\text{Recall} = \frac{TP}{TP + FN}.$$

The F1 score is a balanced metric to capture both Precision and Recall, its specifically the harmonic mean of each.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Table 5.1: Precision, Recall and F1 metric values for each dataset from 60-65min.

Files	Precision	Recall	F1
a01	0.97	0.906	0.937
a12	0.943	0.513	0.664
a13	0.966	0.922	0.944
x04	0.992	0.922	0.956
a03	0.923	0.913	0.918
b01	0.97	0.898	0.932
b02	1.0	0.252	0.402

In Table 5.1 we can see that the model has strong Precision scores from 0.92 - 1.0, however the Recall scores aren't great, with some as little as 0.25, suggesting the model is precise, that is when it recognises a sample as part of a QRS complex it often is, but isn't as strong at locating all samples that are apart of a QRS complex.

The performance above is tested against recognising samples that are apart of QRS complexes, for this project we are more focused on identifying the occurence of a complex. We convert the predicted samples to identify occurence of complexes, by recognising clusters of 1s including patterns such as 1101 as a peak and choosing the middle value as the location. In Table 5.2, we calculate the proportion of matching peaks, along with number of hallucinated and missed peaks for this approach. The results show strong performance, with very few hallucinated or missed peaks, and a large proportion of peaks matched across all datasets.

Table 5.2: Proportion of peaks found, peaks hallucinated and peaks missed by model for each dataset from 60-65min.

Files	Matching	Hallucinated	Missed
a01	1.0	2.0	0.0
a12	0.997	10.0	1.0
a13	0.984	8.0	7.0
x04	1.0	9.0	0.0
a03	1.0	1.0	0.0
b01	0.997	2.0	1.0
b02	0.915	0.0	27.0

5.3 Respiratory Estimation

In Section 5.2 we saw very promising signs for the performance of the Random Forest model, so we decide to explore its ability to detect trends when using RSA with FFT implementation, as we saw that was the most effective approach. In Figure 5.1, we can see that the model is able to predict distinct trends in the expected Respiratory Rate range in x04, b01 and a13 along with less distinct pattern in a01. This shows promising signs for the viability of a RSA based Respiratory Rate estimation method, implementing machine learning to detect peaks.

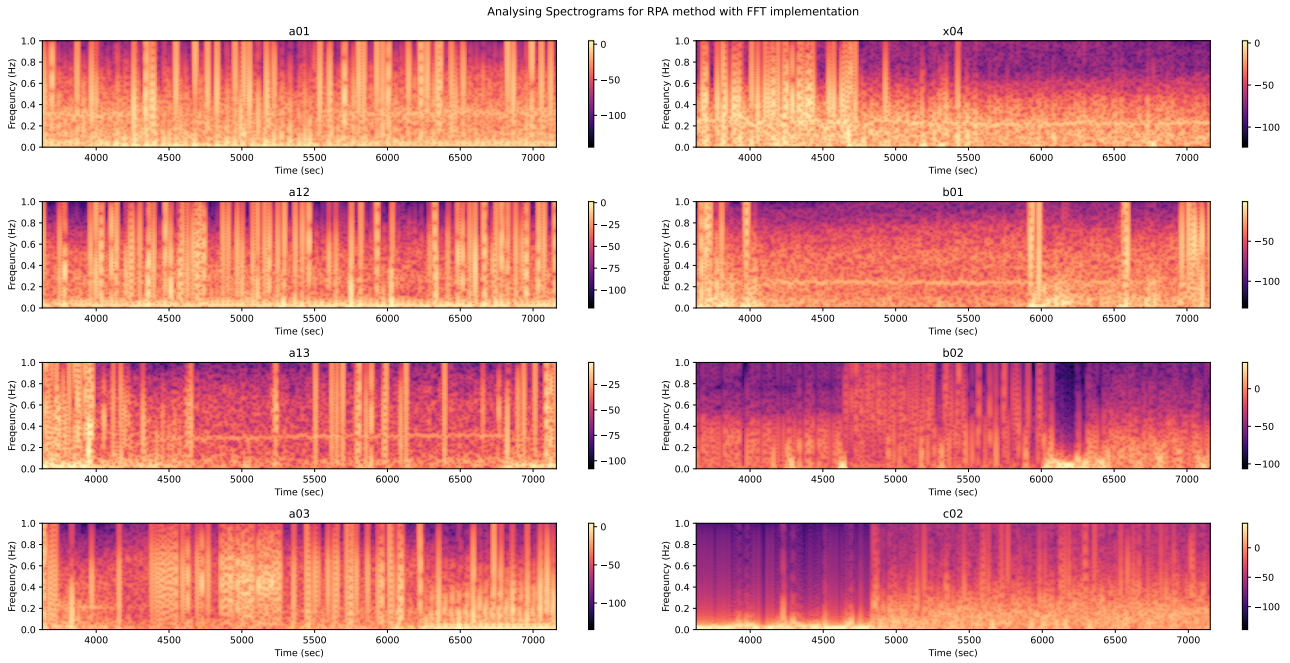


Figure 5.1: Spectrogram comparison across datasets for RSA method with FFT implementation with peaks identified with the Model.

In order to ensure that the trends above weren't artifacts of the Machine Learning Model, we plot the RSA waveforms with peak data from the Jump Algorithm, Annotation Algorithm and Machine Learning Model, as shown in Figure 5.2.

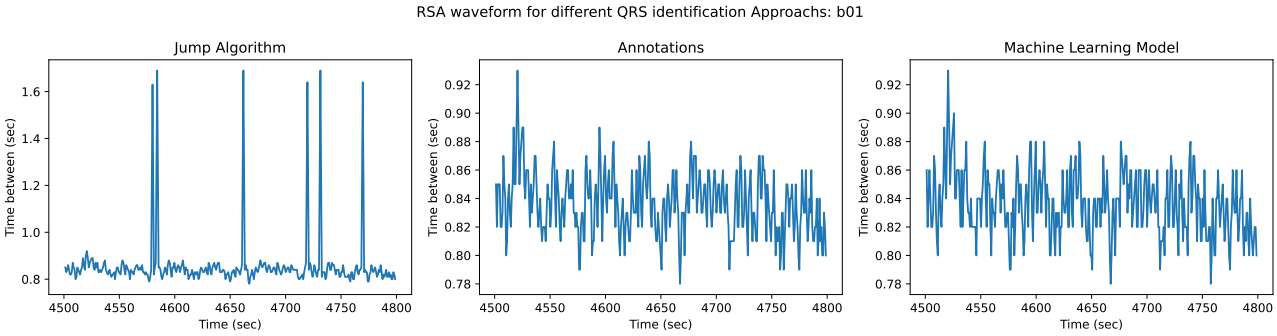


Figure 5.2: Plot comparing RSA waveform utilising different peak identifcation methods.

In Figure 5.2 we can see the clear wave like behaviour of the RSA data for all three methods, validating our spectrum approach, we can see a few artificats in the jump waves, however all three have similiar form. This is a strong indication that the trends in Figure 5.1 aren't an artificat as the plot illustrates the wave in a range where a strong trend is identified, and matches closely to that of the annotations, however this trend isn't seen in the annotations illustrating the ability of the model to highlight trends.

6 Discussion, Conclusion and Reflection

6.1 Discussion

6.1 Peak Detection Algorithm

We saw that developing algorithms with fixed either size or time interval thresholds, weren't reliable options for QRS detection due to the varied morphology of ECG signals. One way in which we attempted to overcome this is by using a Jump Detection, that picked up on a large increase in peaks detected, and then moved back to choose the peaks identified before this Jump, we saw good potential with this algorithm, as it was able to match a large proportion of peaks on 7 of the 8 samples (0.967 - 1.0). Due to the algorithms reliance on amplitude differences, we saw it was less capable on sample b02 which we saw had very small relative difference between QRS peaks, indicating that the algorithm needs to be made more robust against this kind of ECG behaviour. Another limitation of this algorithm was that it was capable of hallucinating a large number of peaks, e.g 538 in dataset x04, suggesting that future improvements implement a mechanism to detect and remove these. Its important to note that the performance of all algorithms were evaluated against annotation files that may themselves have error, as indicated by testing the spectrogram of the annotations files, which showed near identical behaviour to the Jump Algorithm peaks for the RSA data, suggesting that the Algorithm may be sufficient for implementing the RSA method, however when utilising the RPA method the algorithm produces trends not seen in the annotations, supporting the possibility that the annotations contained errors or that trend in RPA may be an artifact. Further due to the sheer size of the data, any errors or issues with the algorithm identified are hard to diagnose as it likely requires visual inspection to see what the algorithm is picking up, however we can only visually inspect small piece of ECG signal at a time, limiting the ability to troubleshoot issues. We also trained a Random Forest Model to identify peaks against the annotation files, this showed very positive results, however would require more validating and testing to ensure the robustness of its performance.

6.1 Respiratory Rate Methods

The first step in developing Respiratory Rate detection algorithms was in choosing the appropriate method for spectrum analysis of the non-uniform time series data, we evaluated the Discrete Fourier Transform with interpolation and the LombScargle Method. We saw that the interpolation appeared to be very accurate, however we were limited in our ability to test this as we were unable to find an appropriate quantative measure due to using discrete data where interpolation was designed to go through the raw data values posing a challenge of detecting differences using usual methods such as MSE which measure the difference in true vs interpolated only at the data points, which would be roughly zero in this case. We relied on visual confirmation, which was limited by only being able to view small snippets of ECG signal at a time. We decided to focus on the FFT based method as it produced clearer Spectrograms in a less computationally expensive way compared to the LombScargle method, however we note this is based on the assumption of the effectivity of the interpolation which is hard to truly verify. One possible way of measuring the accuracy of the interpolation is by using spectrums which was the data we were interested, however as there is no true frequencies to test against, and the LombScargle method had inherit statistical error reducing its validity to produce true frequencies to test against, this is also something we were unable to measure.

When analysing Spectrograms to identify dominate trends the Respiratory related frequency ranges, we saw that the RPA method produced a stronger dominate frequency when it was able to detect one, though this wasn't seen in the annotation files creating doubt about its validity. The RSA method was able to pick up on sections of dominate frequencies in samples that RPA was unable. 7 of the 8 tested datasets did not having a visually obvious pattern in the Spectrogram, indicating that these methods may be limited by the effectivity of the QRS complex identification algorithm, however we did see for the RSA method that using the annnotation files to detect peaks produced the same results suggesting that the methods may only be able to detect Respiratory trends in limited situtation, or that the files themselves contained errors. Another consideration is ECG signals are sensitive to noise and artificats, this would then pushforward onto the Respiratory waveform creating noise in this wave where the analyse is being done, possibly hiding the Respiratory rate frequency. It seems that both methods require cleaner data and possibly more precise QRS identification algorithms then was used in this project, indicating the importance of robust QRS complex identification and clear ECG signals. Despite this, we saw promising signs for the effectiveness of these methods as we saw that the methods were able to pick up on a dominate frequency pattern on some dataset; with the RSA method appearing to be the most viable option due to its ability to detect dominate frequencies with less precise data, as its likely less sensitive to noise as it relies on time between QRS complexes not precise amplitudes. Further, we saw when using the peaks identified by the

Random Forest Model additional trends were seen in the Spectrogram in the expected Respiratory range that didn't appear with the other methods, which considering its effectiveness when evaluated against annotation files goes to highlight the sensitivity of the Spectrograms. The existence of these trends provides additional support that under the correct conditions the RSA method may be able to measure Respiratory Rate. However this was on the same datasets it was trained on (though different time intervals), and thus requires further validation and testing.

6.2 Conclusion

Its clear this project has encountered a number of issues that are common when exploring ECG data for estimation of Respiratory rates, specifically ECGs sensitivity to noise and the reliability of QRS complex identification. We developed a basic algorithm model for identifying QRS complexes based upon a Processing Phase, and then Identification Phase using a adaptive height threshold based upon a minimum numbers of peaks to be identified (i.e peaks per sec), we tested the performance of this algorithm against the QRS annotation files, noting that different time interval thresholds worked for different datasets. We then produced extensions of this algorithm to improve the performance, concluding with a Jump Detection Based Algorithm that produces the best performance when evaluated against QRS annotation files whilst also being able to reproduce the same RSA spectrograms, at the sacrifice of time complexity. This algorithm is still reliant on relative amplitude difference and requires further improvements to deal with the case of small relative peak amplitude differences. We evaluated the RSA and RPA Respiratory Rate estimation methods, using spectrograms as the predomiant tool with the goal of identifying a dominate frequency, that persitted across time, that sits in the expected frequency range of Respiratory Rate which could potentially represent it. We began by evaluating two methods for spectrum analysis of the non-uniform time series, a Discrete Fourier Transform with interpolation and LombScargle, whilst we were unable to quantitatively determine which was more accurate, the FFT based method was more efficient and able to produce clearer spectrograms thus we focused on it. In analysing the RPA and RSA method, it was clear the sensitivity of each method to the accuracy of the data, however results indicated the RSA method may be more viable as it was able to detect patterns with less accurate data, that weren't seen in the RPA method. We saw that a Random Forest Model trained to identify QRS peaks, when implemented alongside the RSA method showed positive signs of being able to detect possible Respiratory trends, providing further support that RSA may be a viable method.

6.3 Area for future research

We suggest the following directions for extending the work of this Project.

Peak Detection Algorithm:

- Implement statistical methods to determine false detection or missed detection of RR peaks, by identifying small or large gaps in RR peak intervals based off the median (or mean) interval size. When detected remove, or adjust threshold.
- Continue investigating the Random Forest and other classification models, for QRS peak detection.

Respiratory Rate Methods:

- Design quantative measures to estimate the error of the interpolation method to provide further validity of this FFT based method.
- Testing methods with other known QRS complex algorithms, e.g Pan Tompkins. Focusing on the RSA method implemented with FFT as that showed the most promise in this project.
- Implement unsupervised learning techniques to identify patterns that aren't visually apparent in the Spectrogram that may correspond to Respiratory Rate.
- Determing appropriate ways of further cleaning data before Spectrograms are produced.

6.4 Reflection

This project was my first time completing a data based project/research, coming in I had no formal experience with python and had only taken one programming suject on C and hadn't taken any data science subjects. I have learnt a lot working on this project, both about completing data based research and in developing technical skills. Looking back at how I approached this project at the start, I now realise I didn't spend enough time initially developing my methodology, e.g finding appropriate measures of accuracy and dived too fast into producing code; for example when testing algorithm I initially only was evaluating proportion of peaks, not looking into hallucinations, matching etc. I now understand the importance of having a clear approach in mind, and validating each assumption you make and being able to test quantitatively your approachs, for example I didn't think to check against the annotation files the trends that would be identified by the spectrograms until after doing weeks of work utilising the other algorithms trying to highlight trends that weren't there. Additionally, I have learnt lots about data processing, focusing largely on time series data and understanding the importance of being able to get data into a workable format where the features you want to explore are clear, and minimally affected by noise; for example using frequency filters to remove noise. Further I have developed a clearer picture on how to approach data based research, including dealing with the challenges of noise, the importance of processing and cleaning data, validating assumptions with quantative results and of visualisation etc. On the technical side, I feel I have developed the fundamentals of python providing me with the basics going forward to learn other programming based skills such as machine learning. I have gained an appreciation for algorithm design, the importance of understanding time complexity and memory usages and being able to develop metrics for evaluating performace. Further, I have developed a strong understand in time series analysis, specifically in utilising spectrums and spectrograms to both filter and learn from the data. I have also learnt the importance of proper documentation and coding practice including when producing plots throughout development, ensuring that time is not wasted cleaning up code or plots at the end in preparation for use of others.

Bibliography

[1] Chapter 1 - normal electrocardiogram: Origin and description. In B. Surawicz and T. K. Knilans, editors, *Chou's Electrocardiography in Clinical Practice (Sixth Edition)*, pages 1–28. W.B. Saunders, Philadelphia, sixth edition edition, 2008. URL: https://www.sciencedirect.com/science/article/pii/B9781416037743100012, doi:10.1016/B978-141603774-3.10001-2.

[2] American Lung Association Editorial Staff. Understanding vital signs: The importance of your respiratory rate. URL: https://www.lung.org/blog/respiratory-rate-vital-signs.

[3] S. R. Braun. Respiratory rate and pattern. In H. K. Walker, W. D. Hall, and J. W. Hurst, editors, *Clinical Methods: The History, Physical, and Laboratory Examinations*, chapter 43. Butterworths, Boston, 3 edition, 1990. URL: https://www.ncbi.nlm.nih.gov/books/NBK365/.

[4] C. Chourpiliadis and A. Bhardwaj. Physiology, respiratory rate, 2022. Updated September 12, 2022. In: StatPearls [Internet]. Treasure Island (FL): StatPearls Publishing. URL: https://www.ncbi.nlm.nih.gov/books/NBK537306/.

[5] Cleveland Clinic. Sinus arrhythmia: Causes, symptoms and treatment. URL: https://my.clevelandclinic.org/health/diseases/21666-sinus-arrhythmia.

[6] M. A. Z. Fariha, R. Ikeura, S. Hayakawa, and S. Tsutsumi. Analysis of pan-tompkins algorithm performance with noisy ecg signals. *Journal of Physics: Conference Series*, 1532(1):012022, 2020. doi:10.1088/1742-6596/1532/1/012022.

[7] E. Kavlakoglu. What is random forest?, 2021. URL: https://www.ibm.com/think/topics/random-forest.

[8] L. Mirmohamadsadeghi and J.-M. Vesin. Respiratory rate estimation from the ecg using an instantaneous frequency tracking algorithm. *Biomedical Signal Processing and Control*, 14:66–72, 2014. URL: https://www.sciencedirect.com/science/article/pii/S1746809414001001, doi:10.1016/j.bspc.2014.07.003.

[9] A. Nicolò, C. Massaroni, E. Schena, and M. Sacchetti. The importance of respiratory rate monitoring: From healthcare to sport and exercise. *Sensors*, 20(21):6396, Nov. 2020. doi:10.3390/s20216396.

[10] J. Pan and W. J. Tompkins. A real-time qrs detection algorithm. *IEEE Transactions on Biomedical Engineering*, 32(3):230–236, 1985. doi:10.1109/TBME.1985.325532.

[11] T. Penzel et al. The apnea-ecg database. In *Computers in Cardiology*, volume 27, pages 255–258, 2000. URL: https://www.physionet.org/content/apnea-ecg/1.0.0/, doi:10.1109/CIC.2000.898505.

[12] PhysioNet. Wfdb applications guide. URL: https://www.physionet.org/physiotools/wag/.

[13] T. Pollard, B. E. Moody, L.-w. H. Lehman, B. J. Gow, C. Fernandes, C. Xie, A. Johnson, R. G. Mark, and T. Heldt. PhysioNet as a global platform for biomedical research. *Nature Health*, 1(8):792–795, Aug. 2026. doi:10.1038/s44360-026-00096-z.

[14] SciPy Developers. scipy.signal.lombscargle. https://docs.scipy.org/doc/scipy-1.18.0/reference/generated/scipy.signal.lombscargle.html, 2026. SciPy v1.18.0 documentation.

[15] J. Valerian. The art of scientific computation: Respiratory rates. Technical report, Aug. 2026.

[16] J. T. VanderPlas. Understanding the lomb–scargle periodogram. *The Astrophysical Journal Supplement Series*, 236(1):16, May 2018. doi:10.3847/1538-4365/aab766.

[17] Victor Chang Cardiac Research Institute. What your resting heart rate says about you, 2026. URL: https://www.victorchang.edu.au/resting-heart-rate.

[18] Wikipedia contributors. Electrocardiography. URL: https://en.wikipedia.org/wiki/Electrocardiography.

[19] World Health Organization. Pneumonia in children. URL: https://www.who.int/news-room/fact-sheets/detail/pneumonia.

[20] M. Zechmeister and M. Kürster. The generalised lomb–scargle periodogram: A new formalism for the floating-mean and keplerian periodograms. *Astronomy & Astrophysics*, 496(2):577–584, Mar. 2009. doi:10.1051/0004-6361:200811296.